

Ingestão de Replies

Alterações no webhook de eventos para persistir mensagens completas e habilitar visualização em chat no app.

Contexto

Hoje o app **Cold Email Pro** mostra apenas o status de cada lead (sent / opened / replied / bounced), sem o conteúdo das trocas. Vamos construir uma **visualização em chat** mostrando a thread completa entre nós e o lead — ida e volta, com timestamp e corpo de cada mensagem.

Para isso precisamos que o N8N passe a persistir cada mensagem (enviada e recebida) como linha individual numa nova tabela `email_messages`, além do update de status já existente.

Estado atual

Workflow responsável: [Email]Webhook eventos e recebimento (id NkZ06yq9LeKVBnbs).

Ele recebe eventos do provider (SmartLead / Resend / etc) e tem um `Switch` com 4 branches: `Resposta`, `Bounced`, `Aberto`, `Entregue`. Para cada um, um `Set` monta o payload e um nó Supabase faz `update` na tabela `emails`.

PROBLEMA

O ramo **Resposta** (`Reposta` → `Reposta_staus`) atualiza o status do lead para `replied`, mas **não está salvando o corpo da resposta**. Verificação feita no banco em 2026-04-23: de 1.203 leads, apenas 7 têm status `replied` e **nenhum** tem `response_content`, `reply_we_got` ou `reply_time` preenchido.

Objetivo

1. Criar tabela normalizada `email_messages` (uma linha por mensagem, inbound ou outbound).
2. Ajustar o workflow de ingestão de eventos para **inserir** em `email_messages` além de atualizar `emails.status`.
3. Ajustar o workflow de **envio** ([email] Send Email pt2 , id jhzBrpA2g5mY0Mon) para **inserir** em `email_messages` toda vez que um email sai.

Com isso o app consegue renderizar a thread ordenada por data.

1 · Schema da nova tabela

A migration será aplicada no Supabase pela equipe de dev (não é o seu trabalho). Mas você precisa saber o shape para montar os `insert` :

```

-- Aplicado no Supabase
CREATE TABLE email_messages (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  email_id uuid NOT NULL REFERENCES emails(id) ON DELETE CASCADE,

  -- Thread tracking (vem do provider)
  thread_id text,
  provider_message_id text,
  in_reply_to text,

  -- Direção
  direction text NOT NULL CHECK (direction IN ('outbound', 'inbound')),

  -- Conteúdo
  from_address text NOT NULL,
  from_name text,
  to_address text NOT NULL,
  subject text,
  body_text text,
  body_html text,

  -- Metadados
  provider text,          -- smartlead | resend | zapmail | gmail
  sent_at timestamptz NOT NULL,
  received_at timestamptz,

  created_at timestamptz DEFAULT now()
);

```

2 · Payload esperado nos inserts

Mensagem recebida (inbound)

```

{
  "user_id": "<user_id do lead>",
  "email_id": "<uuid do lead na tabela emails>",
  "direction": "inbound",
  "from_address": "jake@intelitechs.com",
  "from_name": "Jake Smith",
  "to_address": "sender@meudominio.com",
  "subject": "Re: Quick question about Intelitechs",
  "body_text": "Obrigado pelo contato! Sim, podemos conversar...",
  "body_html": "<p>Obrigado pelo contato!...</p>",
  "thread_id": "<thread_id do provider, se disponível>",
  "provider_message_id": "<message-id da resposta>",
  "in_reply_to": "<message-id do email que respondeu, se tiver>",
  "provider": "smartlead",
  "sent_at": "2026-04-22T14:33:21Z",    // quando o lead mandou
  "received_at": "2026-04-22T14:33:45Z" // quando chegou pra nós
}

```

Mensagem enviada (outbound)

```
{
  "user_id": "<user_id do lead>",
  "email_id": "<uuid do lead>",
  "direction": "outbound",
  "from_address": "sender@meudominio.com",
  "from_name": "Patrick Almeida",
  "to_address": "jake@intelitechs.com",
  "subject": "Quick question about Intelitechs",
  "body_text": "Hey Jake, saw you're leading...",
  "body_html": "<p>Hey Jake...</p>",
  "provider": "zapmail",
  "provider_message_id": "<message-id que o provider retornou no send>",
  "sent_at": "2026-04-15T15:01:31Z"
}
```

IMPORTANTE

O campo `email_id` é **sempre obrigatório** — sem ele não conseguimos linkar a mensagem ao lead. Para inbound, identificar pelo `from_address` : fazer `SELECT id, user_id FROM emails WHERE email = <from_address>` . Se retornar mais de um (mesmo lead em campanhas diferentes), pegar o mais recente por `date_sent` .

3 · Alterações no workflow `NkZ06yq9LeKVBnbs`

Workflow atual tem este fluxo para cada branch:

```
Webhook → Switch → Set (monta payload) → Supabase (update emails)
```

Novo fluxo para o ramo **Resposta** (inbound):

```
Webhook
↓
Switch (4 branches)
  ↓ [Resposta]
  Set (extrai dados do payload do provider)
  ↓
  Postgres: SELECT id, user_id FROM emails WHERE email = <from_address>
  ↓
  If (achou o lead?)
    ↓ TRUE
    Set (monta payload para email_messages)
    ↓
    Supabase INSERT em email_messages      ← NOVO
    ↓
    Supabase UPDATE em emails (status='replied', reply_time=now)
    ↓
    [Opcional] Trigger de notificação (Telegram/Slack/etc)
```

Passos node-a-node

a) Node novo: **Resolve Lead** (Postgres)

Depois do **Set** de resposta, antes de qualquer update. Query:

```
SELECT id, user_id
FROM emails
WHERE LOWER(email) = LOWER($1)
ORDER BY date_sent DESC NULLS LAST
LIMIT 1;
```

Parameter `$1` = `{{ $json.from_email }}` (ou o path onde o provider manda o email de quem respondeu).

b) Node novo: **If lead exists**

Condição: `{{ $json.id }}` `isNotEmpty`. Se não existir o lead, não insere (evita lixo). Opcional: logar num canal de erro.

c) Node novo: **Build message payload** (Set)

Monta o objeto descrito em "Mensagem recebida (inbound)" acima. Campos que precisam ser extraídos do payload do provider:

CAMPO DESTINO	SMARTLEAD	RESEND	ZAPMAIL
<code>from_address</code>	<code>reply.from_email</code>	<code>data.from</code>	<code>sender</code>

CAMPO DESTINO	SMARTLEAD	RESEND	ZAPMAIL
from_name	reply.from_name	data.from_name	sender_name
to_address	reply.to_email	data.to[0]	receiver
subject	reply.subject	data.subject	subject
body_text	reply.message_body_plain	data.text	body_plain
body_html	reply.message_body_html	data.html	body_html
thread_id	reply.thread_id	data.thread_id	—
provider_message_id	reply.message_id	data.message_id	message_id
sent_at	reply.received_at	data.created_at	received_at

Ajustar os paths conforme o payload real de cada provider — os nomes acima são chute educado; validar com `console.log` nos execution logs.

d) Node novo: **Insert email_messages** (Supabase)

- **Operation:** Create
- **Table:** `email_messages`
- **Fields:** todo o objeto do passo (c), com `direction = "inbound"`, `email_id` e `user_id` vindos do passo (a).

e) Node ajustado: **Reposta_staus** (Supabase — já existe)

Manter, mas garantir que atualiza:

- `status = 'replied'`
- `reply_time = <sent_at do inbound>` (ou `now()` se não tiver)
- `time_we_got_reply = <received_at do inbound>`
- **NÃO** escrever no `response_content` / `reply_we_got` / `our_last_reply` — isso vai pra `email_messages`. Esses campos antigos ficarão deprecated e serão removidos no futuro.

4 · Alterações no workflow de envio `jhzBrpA2g5mY0Mon`

Para o chat mostrar a **nossa** parte da conversa, o workflow de envio também precisa inserir em `email_messages` toda vez que um email sai — seja o inicial, follow-up ou qualquer resposta manual.

Onde adicionar

Hoje o workflow tem nodes `HTTP Request1` (envio direto) e `Zapmail – Enviar Email`, seguidos de `Update a row / Update a row1` que atualizam `emails`. Adicionar um node `Supabase: Insert email_messages` **depois** do envio bem-sucedido e **antes** (ou em paralelo com) o update de `emails`.

```
[envio HTTP/Zapmail] → Insert email_messages (NOVO) → Update a row (emails)
```

Payload para outbound

Igual ao exemplo "Mensagem enviada (outbound)" acima. Notas:

- `email_id` já está no contexto (é o lead que o pt1 passou pro pt2).
- `user_id` também — vem do lead.
- `from_address`, `from_name` = o sender escolhido (está disponível no fluxo depois do node `SENDER_NAME / SENDER_NAME1`).
- `to_address` = email do lead.
- `subject`, `body_text`, `body_html` = o que foi construído nos `Edit Fields` de acordo com o tipo de email.
- `provider_message_id` = ID que o provider retorna na resposta do HTTP de envio (usar `{{ $node["HTTP Request1"].json.id }}` ou similar).
- `sent_at` = `{{ $now }}`.

Also: aplicar ao `[emails] Follow ups ( 0x9tjMCXLxba1LqZ )`

Esse workflow também manda emails (via `Send Email Resend`). Adicionar o mesmo insert em `email_messages` depois do envio.

5 · Testing

Inbound

1. Mandar um email inicial (via app ou manualmente) pra um lead de teste.
2. Responder daquele endereço.
3. Verificar no Supabase: `SELECT * FROM email_messages WHERE email_id = '<lead>' ORDER BY sent_at;`
4. Deve aparecer 1 linha `outbound` (passo 1) e 1 `inbound` (passo 2).
5. A linha inbound deve ter `body_text` e/ou `body_html` preenchido.

Outbound

1. Enviar email via `/dashboard` → Dispatch.

2. Verificar `SELECT * FROM email_messages WHERE direction='outbound' ORDER BY created_at DESC LIMIT 5;`
3. Todos devem ter `body_html`, `subject`, `provider_message_id`.

6 • Checklist

- ☐ Confirmei o payload real do provider (SmartLead ou Resend) rodando um teste e capturando o JSON que chega no Webhook.
- ☐ Adicionei node `Resolve lead` (Postgres SELECT) no ramo Resposta.
- ☐ Adicionei node `If lead exists` que pula quando não acha.
- ☐ Adicionei node `Build message payload` (Set) com os campos da tabela.
- ☐ Adicionei node `Insert email_messages` (Supabase Create) com `direction='inbound'`.
- ☐ Ajustei `Reposta_staus` para atualizar só `status`, `reply_time`, `time_we_got_reply`.
- ☐ No workflow de envio (`jhzBrpA2g5mY0Mon`), adicionei `Insert email_messages` com `direction='outbound'` depois de cada envio bem-sucedido.
- ☐ No workflow de follow-ups (`0x9tjMCXLxba1LqZ`), idem.
- ☐ Testei inbound com um endereço real e validei que veio o corpo no banco.
- ☐ Testei outbound e validei que a mensagem enviada também ficou registrada.
- ☐ Ativei os workflows alterados.

7 • Observações

- A tabela `email_messages` vai crescer rápido (uma linha por envio + uma por reply). RLS será aplicada via `auth.uid() = user_id` — garantir que o `user_id` é sempre preenchido corretamente.
- Os campos antigos da tabela `emails` (`response_content`, `reply_we_got`, `our_last_reply`, `reply_time`, `time_we_got_reply`) vão ficar deprecated. Por ora, manter `reply_time` e `time_we_got_reply` atualizados pra compatibilidade com o app atual. Os 3 de corpo podem parar de ser escritos.
- Se o provider mandar anexos, **não incluir no payload por enquanto** — será adicionado numa segunda fase (nova tabela `email_attachments`).
- Se acontecer de o webhook ser chamado duas vezes para o mesmo `provider_message_id` (duplicação), não é problema hoje, mas no futuro adicionaremos um `UNIQUE` em (`provider`, `provider_message_id`) — o dev do front vai alinhar quando acontecer.